NeuroSky MindWave Mobile Windows SDK

Developer Guide · v2.0.0

Table of Contents

---

1. Overview

The **NeuroSky MindWave Mobile Windows SDK** is a modern C# library that lets you read real-time EEG (electroencephalography) data from a NeuroSky MindWave Mobile headset on Windows 10 or later — with zero dependency on NeuroSky's legacy ThinkGear Connector (TGC) software.

Why this SDK exists

The official NeuroSky SDK requires TGC, a background Windows service, to be running on the user's machine. This creates friction: users must install and start a separate process, troubleshoot port conflicts, and deal with a heavyweight dependency that is difficult to bundle in modern applications.

This SDK eliminates TGC entirely by communicating directly with the MindWave Mobile hardware via the Windows Bluetooth stack (WinRT). Your application talks to the headset directly — no intermediary service, no installer prerequisite beyond .NET 8.

Key features

Feature	Description
No TGC dependency	Communicates with hardware directly via WinRT
BLE + BT Classic	Supports both Bluetooth transports
Developer-selectable transport	`TransportMode.Auto / Ble / BtClassic`
Async stream API	`IAsyncEnumerable<BrainWaveData>` — native `await foreach`

Built-in Simulator	Full data simulation without any hardware
.NET 8 / C# 12	Modern language features, nullable annotations

What you can measure

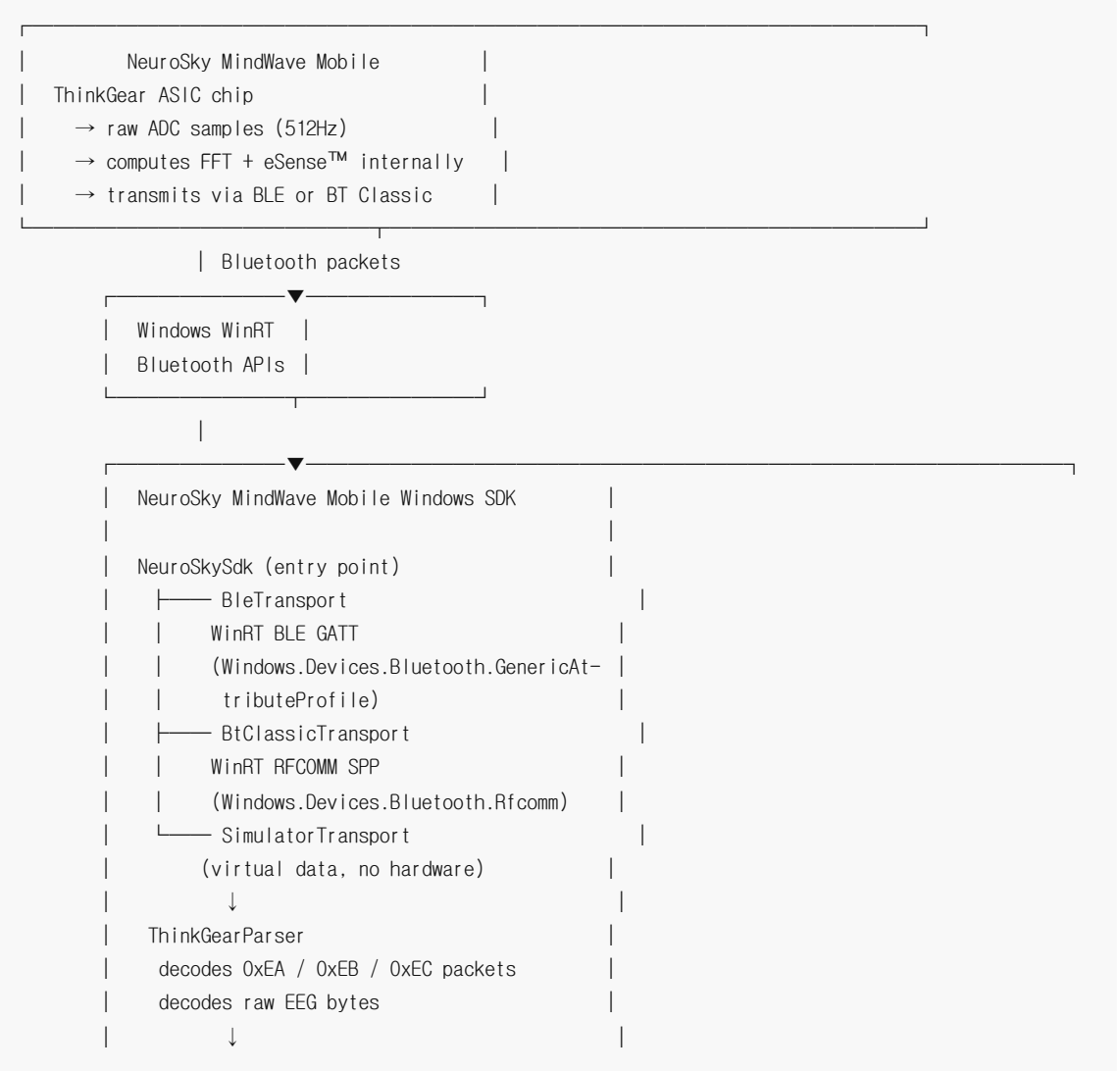
The MindWave Mobile headset contains a single dry electrode on the forehead (FP1 position) and a reference clip on the ear. From this single channel, the ThinkGear ASIC chip on board computes:

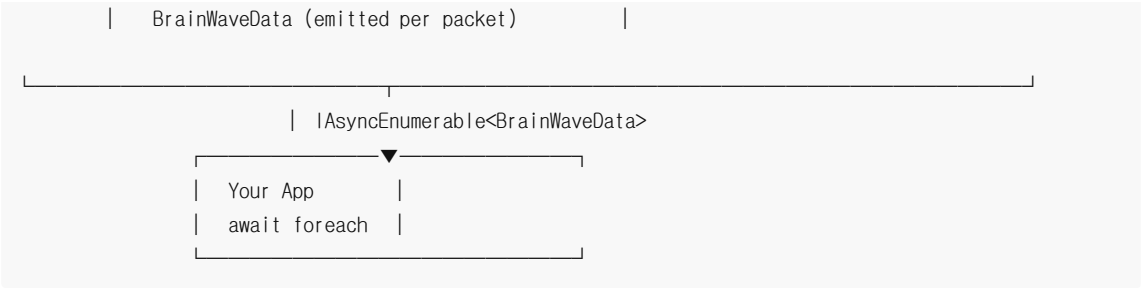
- **Raw EEG waveform** — 512 samples/sec, signed 16-bit values
- **8 frequency band powers** — Delta, Theta, Alpha (Low/High), Beta (Low/High), Gamma (Low/Mid)
- **eSense™ Attention** — NeuroSky's proprietary attention index (0~100)
- **eSense™ Meditation** — NeuroSky's proprietary relaxation index (0~100)
- **Eye blink detection** — intensity 0~255
- **Signal quality** — 0 (perfect contact) to 200 (no signal)

---

2. How It Works — Architecture

Understanding the data flow helps you design your application and debug issues effectively.





BLE vs BT Classic — internal differences

BLE (Bluetooth Low Energy) path: The MindWave Mobile exposes three BLE GATT characteristics:

- `039afff8-...` — eSense data (Attention, Meditation, frequency bands) — SDK subscribes to notifications
- `039afff4-...` — Raw EEG data — SDK subscribes to notifications
- `039affa0-...` — Handshake characteristic — SDK writes command bytes to start data flow

BT Classic (RFCOMM SPP) path: The MindWave Mobile emulates a serial port (Serial Port Profile, UUID `00001101-...`). The SDK opens an RFCOMM socket and reads a continuous byte stream. The ThinkGearParser synchronizes on the `0xAA 0xAA` sync bytes and parses variable-length packets.

Both paths produce identical `BrainWaveData` output. The parsing layer is shared.

---

3. Requirements

System requirements

Component	Minimum version	Notes
Windows	Windows 10 version 1903 (build 18362)	WinRT BLE GATT requires 1903+
.NET runtime	.NET 8.0	Must be installed on target machine
Bluetooth adapter	BLE-capable adapter	For `TransportMode.Ble` or `Auto`
Bluetooth adapter	Classic BT adapter	For `TransportMode.BtClassic`
Device pairing	Not required for BLE	Required for BT Classic

Project requirements

Your application's `.csproj` must target the Windows platform TFM (Target Framework Moniker) to access WinRT APIs:

```
<!-- Required — standard net8.0 will NOT work -->
<TargetFramework>net8.0-windows10.0.19041.0</TargetFramework>
```

The `10.0.19041.0` suffix corresponds to Windows 10 version 2004. This is the minimum build required for stable WinRT BLE GATT support. Targeting this version does **not** prevent the app from running on newer Windows 10/11 builds.

> ***Important:*** *If your project omits the `-windows10.0.19041.0` suffix and targets plain `net8.0`, the WinRT types (`Windows.Devices.*`) will not be available and the SDK will throw `PlatformNotSupportedException` at runtime.*

Supported headset

This SDK is designed and tested for the **NeuroSky MindWave Mobile 2** (sometimes labeled just "MindWave Mobile"). The original MindWave (wired, USB dongle) is not supported. Both BLE and BT Classic modes of the MindWave Mobile 2 are supported.

---

4. Installation

Option A — NuGet Package Manager UI (Visual Studio)

1. Right-click your project in Solution Explorer → **Manage NuGet Packages**
2. Select the **Browse** tab
3. Search for: `NeuroSky.MindWave.Sdk`
4. Click **Install**

Option B — Edit `.csproj` directly (recommended for CI/CD)

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <!-- Windows TFM required for WinRT APIs -->
    <TargetFramework>net8.0-windows10.0.19041.0</TargetFramework>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="NeuroSky.MindWave.Sdk" Version="2.0.0" />
  </ItemGroup>
</Project>
```

Option C — .NET CLI

```
dotnet add package NeuroSky.MindWave.Sdk --version 2.0.0
```

Verify installation

After installing, confirm the package resolves correctly:

```
dotnet restore
dotnet build
```

If you see `CS0246: The type or namespace name 'NeuroSkySdk' could not be found`, check that:

1. The package is listed in `.csproj`
2. `TargetFramework` includes the `-windows` suffix
3. You have added `using NeuroSky.Sdk;` at the top of your file

---

5. Quick Start

The following example demonstrates a complete minimal application that connects to a MindWave Mobile headset and streams EEG data until the user presses Ctrl+C.

```
using NeuroSky.Sdk;
using NeuroSky.Sdk.Transport;

// Step 1: Create the SDK instance.
// NeuroSkySdk is IAsyncDisposable - use 'await using' so it disconnects
// cleanly when the block exits (including on exception or Ctrl+C).
await using var sdk = new NeuroSkySdk();

// Step 2: Subscribe to connection state changes (optional but recommended).
// This fires on Scanning → Connecting → Connected → Disconnected transitions.
// Useful for updating UI or logging.
sdk.StateChanged += (_, state) =>
    Console.WriteLine($"[State] {state}");

// Step 3: Set up graceful cancellation.
// CancellationTokenSource lets you stop the stream cleanly.
// Ctrl+C cancels the token instead of killing the process.
using var cts = new CancellationTokenSource();
Console.CancelKeyPress += (_, e) =>
{
    e.Cancel = true;   // prevent immediate process termination
    cts.Cancel();      // signal the data stream to stop
};

// Step 4: Connect to the headset.
// Replace with your MindWave Mobile's actual MAC address.
// Default mode is Auto: tries BLE first, falls back to BT Classic.
// See Section 14 for how to find your MAC address.
await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF");

// Step 5: Set notch filter for your region (recommended).
// This removes 50Hz or 60Hz power-line noise from raw EEG.
await sdk.SendCommandAsync(NeuroSkyCommand.Notch60Hz);  // Korea / USA
// await sdk.SendCommandAsync(NeuroSkyCommand.Notch50Hz);  // Europe / China

// Step 6: Stream data.
// DataStream() returns IAsyncEnumerable<BrainWaveData>.
// Each iteration yields one parsed EEG packet (~1 per second for
// Attention/Meditation; ~51 per second when raw EEG is enabled).
await foreach (var data in sdk.DataStream(cts.Token))
{
    Console.Clear();
    Console.WriteLine($"Signal Quality : {data.SignalQuality} (PoorSignal={data.PoorSignal})");
    Console.WriteLine($"Attention      : {data.Attention,3}");
    Console.WriteLine($"Meditation     : {data.Meditation,3}");
    Console.WriteLine($"--- EEG Bands ---");
```

```
    Console.WriteLine($"Delta          : {data.Delta}");
    Console.WriteLine($"Theta          : {data.Theta}");
    Console.WriteLine($"Low Alpha      : {data.LowAlpha}");
    Console.WriteLine($"High Alpha     : {data.HighAlpha}");
    Console.WriteLine($"Low Beta       : {data.LowBeta}");
    Console.WriteLine($"High Beta      : {data.HighBeta}");
    Console.WriteLine($"Low Gamma      : {data.LowGamma}");
    Console.WriteLine($"Mid Gamma      : {data.MidGamma}");
}
```

> ***Tip:*** *Replace* `"AA:BB:CC:DD:EE:FF"` *with your headset's Bluetooth MAC address. See Section 14 for step-by-step instructions.*

---

6. Connection Modes (TransportMode)

The `TransportMode` enum gives you explicit control over which Bluetooth protocol the SDK uses to communicate with the MindWave Mobile headset. Different modes suit different development scenarios.

Overview

```
public enum TransportMode
{
    Auto,       // BLE first → BT Classic fallback after 5 seconds (default)
    Ble,        // BLE only — no Windows pairing required
    BtClassic   // BT Classic only — requires Windows Bluetooth pairing
}
```

TransportMode.Auto (default)

```
// Both lines are equivalent:
await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF");
await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF", TransportMode.Auto);
```

How it works:

1. The SDK starts a BLE GATT scan and connection attempt
2. A 5-second internal timeout starts simultaneously
3. If BLE connects within 5 seconds → uses BLE for the session
4. If BLE does not connect within 5 seconds → automatically cancels the BLE attempt and starts a BT Classic (RFCOMM) connection
5. Regardless of which transport is used, `DataStream()` behaves identically

When to use Auto:

- You don't know in advance whether the user has the device paired or not
- You want maximum compatibility across different Windows Bluetooth adapter types
- You are building a general-purpose application and don't want to expose transport details to end users

Tradeoff: In environments where BLE is not supported by the adapter, Auto will always wait 5 seconds before falling back. If BT Classic is your intended transport, use `TransportMode.BtClassic` directly to save that delay.

TransportMode.Ble

```
await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF", TransportMode.Ble);
```

How it works:

The SDK uses WinRT's `BluetoothLEDevice.FromBluetoothAddressAsync()` to connect to the MindWave Mobile via BLE GATT. It then discovers services, subscribes to the eSense and RawEEG characteristics, and sends the handshake command to begin data streaming.

Advantages:

- **No Windows pairing required.** The user does not need to open Bluetooth settings and pair the device. The SDK connects directly using the MAC address.
- **Lower power consumption.** BLE uses significantly less power than Classic Bluetooth, which matters for battery-powered host devices.
- **Cleaner application distribution.** No user setup step is needed.

Disadvantages:

- Requires a BLE-capable Bluetooth adapter (most adapters manufactured after 2012 support BLE)
- May be less reliable in environments with heavy 2.4GHz interference (Wi-Fi routers, microwaves)

When to use Ble:

- Your users should not need to manually pair the device
- You are confident the target machine has a BLE adapter
- You want the smoothest user experience

TransportMode.BtClassic

```
await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF", TransportMode.BtClassic);
```

How it works:

The SDK uses WinRT's `RfcommDeviceService` to open an RFCOMM socket over SPP (Serial Port Profile). The connection behaves like a virtual serial port at 57600 baud. The ThinkGearParser reads the incoming byte stream and synchronizes on the `0xAA 0xAA` sync header.

Before using BT Classic — pair the device first:

1. Open **Settings → Bluetooth & other devices**
2. Click **Add device**
3. Select **Bluetooth**
4. Wait for "**MindWave Mobile**" to appear in the list
5. Click it and follow the pairing prompt (no PIN required)
6. Confirm the device shows as **Paired** in the list

Advantages:

- More stable in environments with BLE interference
- Higher raw throughput capacity (though the MindWave Mobile data rate is low either way)

- Familiar serial-port-style communication internally

Disadvantages:

- Requires a one-time Windows pairing step per machine
- Cannot be paired programmatically; the user must do it manually in Windows Settings

When to use BtClassic:

- You are deploying to a controlled environment where devices are pre-paired by IT
- BLE connectivity is unreliable on the target hardware
- You are integrating with existing BT Classic infrastructure

---

Connection State Flow

Every transport goes through the same state machine:

```
Disconnected
     |
     ▼  ConnectAsync() called
  Scanning
     |
     ▼  Device found
  Connecting
     |          |
     ▼          ▼
  Connected    Error
     |
     ▼  DisconnectAsync() or DisposeAsync()
  Disconnected
```

Subscribe to `StateChanged` to update your UI or logging:

```
sdk.StateChanged += (_, state) =>
{
    switch (state)
    {
        case ConnectionState.Scanning:
            statusLabel.Text = "Searching for MindWave Mobile...";
            break;
        case ConnectionState.Connecting:
            statusLabel.Text = "Connecting to MindWave Mobile...";
            break;
        case ConnectionState.Connected:
            statusLabel.Text = "Connected";
            connectButton.IsEnabled = false;
            disconnectButton.IsEnabled = true;
            break;
        case ConnectionState.Error:
            statusLabel.Text = "Connection failed. Check Bluetooth and try again.";
            break;
        case ConnectionState.Disconnected:
            statusLabel.Text = "Disconnected";
```

```
            connectButton.IsEnabled = true;
            disconnectButton.IsEnabled = false;
            break;
    }
};
```

---

7. EEG Data Model

`DataStream()` yields a `BrainWaveData` object for each packet received from the MindWave Mobile headset. Understanding what each field represents and when it is populated will help you use the data correctly.

Full data model

```
public record BrainWaveData
{
    // Time this packet was received by the SDK (Unix timestamp, milliseconds UTC)
    public long Timestamp { get; init; }

    // Electrode contact quality: 0 = perfect, 200 = no signal
    public int PoorSignal { get; init; }

    // eSense™ Attention index (0~100). 0 means not yet computed or no signal.
    public int Attention { get; init; }

    // eSense™ Meditation index (0~100). 0 means not yet computed or no signal.
    public int Meditation { get; init; }

    // EEG frequency band powers (arbitrary units, relative)
    public int Delta     { get; init; }   // 0.5~2.75 Hz
    public int Theta     { get; init; }   // 3.5~6.75 Hz
    public int LowAlpha  { get; init; }   // 7.5~9.25 Hz
    public int HighAlpha { get; init; }   // 10~11.75 Hz
    public int LowBeta   { get; init; }   // 13~16.75 Hz
    public int HighBeta  { get; init; }   // 18~29.75 Hz
    public int LowGamma  { get; init; }   // 31~39.75 Hz
    public int MidGamma  { get; init; }   // 41~49.75 Hz

    // Raw ADC samples: 10 samples per BLE packet, 512Hz total
    // Values are signed 16-bit integers: -32768 to +32767
    public IReadOnlyList<int> RawEeg { get; init; }

    // Eye blink intensity: 0 = no blink, 1~255 = blink detected
    public int EyeBlink { get; init; }

    // Derived from PoorSignal — convenience enum (Good / Fair / Poor / NoSignal)
    public SignalQuality SignalQuality { get; }
}
```

Data update rates

Not all fields update at the same rate. The MindWave Mobile sends different packet types at different intervals:

Field(s)	Update rate	Notes
`PoorSignal`	~1 Hz	Updated every packet
`Attention`, `Meditation`	~1 Hz	eSense™ computed once per second
`Delta` through `MidGamma`	~1 Hz	FFT computed once per second
`RawEeg`	512 Hz	10 samples per BLE notify (~51 packets/sec)
`EyeBlink`	Event-driven	Only non-zero when blink is detected

> ***Important:*** *When `RawEeg` packets arrive, `Attention`, `Meditation`, and frequency band fields will be `0` in that `BrainWaveData` object — they are only populated in the eSense packet (which arrives separately, once per second). Filter by checking which fields are non-zero, or handle each packet type independently.*

Working with timestamps

`Timestamp` is the SDK-side receive time in Unix milliseconds (UTC):

```
var receivedAt = DateTimeOffset.FromUnixTimeMilliseconds(data.Timestamp);
Console.WriteLine($"Packet received at: {receivedAt:HH:mm:ss.fff}");
```

To align EEG samples with external events (e.g., stimulus timing in a BCI experiment), record the `Timestamp` alongside each data point and synchronize using a shared clock reference.

---

8. EEG Frequency Bands Explained

The MindWave Mobile's ThinkGear chip performs a Fast Fourier Transform (FFT) on the raw EEG waveform and outputs the power in 8 frequency bands. These are the same bands standard in clinical EEG research.

What the values mean

The frequency band values are **relative power** in arbitrary units. They are not calibrated to physical units (μV²/Hz). This means:

- **You cannot compare values across individuals or sessions** in absolute terms
- **You can compare values within a session** — e.g., "Delta increased by 30% after eyes closed"
- **Ratios are more meaningful than raw values** — e.g., `theta / (alpha + beta)` for attention estimation

Band reference table

Property	Greek	Range	Hz	Typical mental states
`Delta`	δ	Slow	0.5~2.75 Hz	Deep sleep, healing, unconscious processing. High delta while awake may indicate fatigue or poor signal.
`Theta`	θ	Slow	3.5~6.75 Hz	Drowsiness, light sleep, daydreaming, creative insight, REM. Common in deep meditation.

LowAlpha	α low	Medium	7.5~9.25 Hz	Relaxed, calm, unfocused. Increases when eyes are closed and mind is at rest.
HighAlpha	α high	Medium	10~11.75 Hz	Eyes-closed relaxation. Suppressed by visual attention (alpha blocking).
LowBeta	β low	Fast	13~16.75 Hz	Alert, focused attention, active thinking. The "work" band.
HighBeta	β high	Fast	18~29.75 Hz	Intense cognition, anxiety, high arousal, stress. Elevated during difficult mental tasks.
LowGamma	γ low	Very fast	31~39.75 Hz	Higher-order cognition, cross-modal sensory binding, perception.
MidGamma	γ mid	Very fast	41~49.75 Hz	Intense concentration, feature binding, Tibetan monk meditation studies show elevated gamma.

eSense™ Attention and Meditation

These are NeuroSky's **proprietary processed values** — not raw frequency powers. They are computed by NeuroSky's closed-source algorithm running on the ThinkGear chip itself. The SDK receives these pre-computed values directly.

Attention (0~100):

Reflects the level of mental focus or concentration. The algorithm combines Beta and Gamma activity with suppression of Alpha and Delta/Theta.

Range	Interpretation
0	Not computed (no signal, or headset just turned on)
1~40	Low attention — distracted, relaxed, or wandering mind
40~60	Neutral / baseline
60~80	Moderate attention — engaged in a task
80~100	High attention — strong active focus

Meditation (0~100):

Reflects the level of mental calmness or relaxation. Primarily correlated with Alpha wave activity. High attention and high meditation can coexist (calm focus).

Range	Interpretation
0	Not computed (no signal, or headset just turned on)
1~40	Low meditation — active thinking, stress, or poor signal
40~60	Neutral / baseline
60~80	Moderate relaxation

80~100	Deep calm — strong meditation state

> ***Note:*** *eSense™ values require a stable signal (PoorSignal = 0 or low) and take about 10~20 seconds to stabilize after the headset is put on. Values of 0 at startup are normal.*

Raw EEG

When Raw EEG is enabled (via `StartRawEeg` command), `RawEeg` is populated with 10 signed 16-bit ADC samples per packet at 512 Hz:

```
// Enable raw EEG after connecting
await sdk.SendCommandAsync(NeuroSkyCommand.StartRawEeg);

await foreach (var data in sdk.DataStream(cts.Token))
{
    foreach (var sample in data.RawEeg)
    {
        // Each sample: -32768 to +32767
        // At 512 Hz, 10 samples per packet ≈ 51 packets per second
        PlotSample(sample);
    }
}
```

Raw EEG is useful for custom signal processing, FFT analysis with different window sizes, artifact detection, or research applications. It is disabled by default to reduce Bluetooth bandwidth.

---

9. Signal Quality

Signal quality is the most critical factor for usable EEG data. Before trusting Attention, Meditation, or frequency band values, always verify that the signal quality is acceptable.

PoorSignal value

The `PoorSignal` field is a raw contact quality indicator from the ThinkGear chip:

- **0** — perfect electrode contact, clean signal
- **1~50** — some noise present, but data is usable
- **51~199** — poor contact, data is unreliable
- **200** — no contact detected (headset not worn, or electrode lifted)

SignalQuality enum

The SDK maps `PoorSignal` to a `SignalQuality` enum for convenience:

`SignalQuality`	`PoorSignal` **range**	**Data reliability**
`Good`	0	Excellent — use all data freely
`Fair`	1~50	Acceptable — minor noise, eSense still valid
`Poor`	51~199	Unreliable — reject or flag data
`NoSignal`	200	No data — headset not worn

Recommended signal quality check pattern

```
await foreach (var data in sdk.DataStream(cts.Token))
{
    // Always check signal quality before using the data
    switch (data.SignalQuality)
    {
        case SignalQuality.NoSignal:
            Console.WriteLine("Please put on the MindWave Mobile headset.");
            Console.WriteLine("Make sure the sensor touches your forehead.");
            continue;  // skip this packet entirely

        case SignalQuality.Poor:
            Console.WriteLine($"Weak signal (PoorSignal={data.PoorSignal}).");
            Console.WriteLine("Try adjusting the headset or moistening the sensor.");
            // Optionally still collect data but mark it as low-quality
            RecordDataWithFlag(data, lowQuality: true);
            continue;

        case SignalQuality.Fair:
        case SignalQuality.Good:
            // Data is usable - proceed normally
            ProcessData(data);
            break;
    }
}
```

Tips for improving signal quality

1. **Moisten the sensor tip** — a small amount of water or electrode gel on the forehead sensor greatly improves conductance
2. **Remove glasses** — metal frames can create interference
3. **Clean the forehead** — remove sunscreen, makeup, or sweat
4. **Adjust headset position** — the sensor should be centered on the forehead at FP1 (just above the left eyebrow)
5. **Check the ear clip** — the reference ear clip must make solid contact with the earlobe; a thin or folded earlobe can cause issues
6. **Wait 20~30 seconds** after putting on the headset for values to stabilize

---

10. Commands

After connecting to the MindWave Mobile headset, you can send control commands to configure its behavior. Commands are sent via `SendCommandAsync(byte)`.

Notch filter

EEG signals are very low amplitude (microvolts) and are easily contaminated by electromagnetic interference from AC power lines. The MindWave Mobile's ThinkGear chip has a built-in notch filter that can attenuate power-line noise.

Which frequency to use:

Region	Grid frequency	Command
Korea	60 Hz	`NeuroSkyCommand.Notch60Hz`
USA / Canada	60 Hz	`NeuroSkyCommand.Notch60Hz`
Japan	50/60 Hz mixed	`NeuroSkyCommand.Notch60Hz` (East) / `Notch50Hz` (West)
Europe	50 Hz	`NeuroSkyCommand.Notch50Hz`
China	50 Hz	`NeuroSkyCommand.Notch50Hz`
Australia / UK	50 Hz	`NeuroSkyCommand.Notch50Hz`

```
// Set immediately after connecting, before reading data
await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF");
await sdk.SendCommandAsync(NeuroSkyCommand.Notch60Hz);  // or Notch50Hz
```

Without the notch filter, you may see a large 50Hz or 60Hz sine wave artifact in raw EEG data, and elevated Beta activity in the frequency band output.

Raw EEG streaming

Raw EEG is **disabled by default** to minimize Bluetooth bandwidth usage. Enable it only when your application needs the raw waveform:

```
// Enable raw EEG after connecting
await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF");
await sdk.SendCommandAsync(NeuroSkyCommand.StartRawEeg);

// ... stream and process data ...

// Disable when done (optional — automatically stops on disconnect)
await sdk.SendCommandAsync(NeuroSkyCommand.StopRawEeg);
```

When raw EEG is enabled, the `RawEeg` list in each `BrainWaveData` packet is populated with 10 samples. When disabled, `RawEeg` is always empty.

eSense control

eSense (Attention and Meditation) is enabled by default. You can toggle it:

```
// Disable eSense if you only need raw EEG (reduces processing load on chip)
await sdk.SendCommandAsync(NeuroSkyCommand.StopEsense);

// Re-enable
await sdk.SendCommandAsync(NeuroSkyCommand.StartEsense);
```

All commands reference

Constant	Raw byte	Description

`NeuroSkyCommand.Notch60Hz`	0x1C	Notch filter at 60 Hz
`NeuroSkyCommand.Notch50Hz`	0x1B	Notch filter at 50 Hz
`NeuroSkyCommand.StartRawEeg`	0x15	Begin raw EEG transmission
`NeuroSkyCommand.StopRawEeg`	0x16	Stop raw EEG transmission
`NeuroSkyCommand.StartEsense`	0x17	Enable eSense processing
`NeuroSkyCommand.StopEsense`	0x18	Disable eSense processing

---

11. Simulator — Develop Without Hardware

The `SimulatorTransport` generates realistic synthetic EEG data without any physical MindWave Mobile headset. It implements the same `ITransport` interface as the real transports, so your application code remains unchanged between development and production.

Why use the Simulator

- **No hardware required** — develop and test UI, data pipelines, and business logic before the headset arrives
- **Predictable data** — use `Focused` mode to always produce high-attention data for UI testing without needing to concentrate
- **Edge case testing** — use `PoorSignal` mode to test how your app handles bad signal conditions
- **CI/CD pipelines** — run automated tests without Bluetooth hardware on build servers

Basic usage

```
using NeuroSky.Sdk.Simulator;

// Create simulator
var simulator = new SimulatorTransport();

// Choose simulation mode
simulator.SetMode(SimulatorTransport.Mode.Focused);

// Connect (any string is accepted as the address)
await simulator.ConnectAsync("simulator");

// Stream data - works exactly like the real SDK
await foreach (var data in simulator.DataStream(cts.Token))
{
    Console.WriteLine($"[SIM] Attention: {data.Attention}, Meditation: {data.Meditation}");
}
```

Simulator modes

Mode	Attention	Meditation	PoorSignal	Use case

`Random`	0~100 (random each tick)	0~100 (random each tick)	0	General integration testing, unpredictable data
`Focused`	70~100	40~60	0	Test UI elements that respond to high attention
`Relaxed`	20~50	70~100	0	Test UI elements that respond to high meditation
`PoorSignal`	0	0	200	Test NoSignal handling, reconnect logic, error states

Switching modes at runtime

```
// Start in poor signal, then transition to focused state
var simulator = new SimulatorTransport();
simulator.SetMode(SimulatorTransport.Mode.PoorSignal);
await simulator.ConnectAsync("simulator");

// After 5 seconds, switch to focused mode
await Task.Delay(5000);
simulator.SetMode(SimulatorTransport.Mode.Focused);
```

Dependency injection pattern

Structure your code so the transport is swappable:

```
// ITransport is the common interface - both NeuroSkySdk and SimulatorTransport implement it
ITransport transport;

if (args.Contains("--simulator"))
{
    var sim = new SimulatorTransport();
    sim.SetMode(SimulatorTransport.Mode.Focused);
    await sim.ConnectAsync("simulator");
    transport = sim;
}
else
{
    var sdk = new NeuroSkySdk();
    await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF");
    transport = sdk;
}

// Application code is the same regardless of transport
await foreach (var data in transport.DataStream(cts.Token))
{
    ProcessData(data);
}
```

---

12. Error Handling & Reconnection

EEG applications often run for extended periods. Robust error handling and automatic reconnection are essential for production use.

Connection errors

`ConnectAsync` can throw in several scenarios:

```
try
{
    await sdk.ConnectAsync("AA:BB:CC:DD:EE:FF", TransportMode.Ble);
}
catch (OperationCanceledException)
{
    // The CancellationToken was triggered before connection completed,
    // OR the 5-second Auto timeout fired (Auto mode only).
    Console.WriteLine("Connection cancelled or timed out.");
}
catch (UnauthorizedAccessException)
{
    // BT Classic: device is not paired in Windows Settings.
    // BLE: Bluetooth adapter is disabled or no permission.
    Console.WriteLine("Bluetooth access denied. Check adapter and permissions.");
}
catch (Exception ex)
{
    // Hardware not found, adapter missing, or other OS-level Bluetooth error.
    Console.WriteLine($"Connection failed: {ex.Message}");
}
```

Stream disconnection and auto-reconnect

`DataStream()` ends (the `await foreach` loop exits) when the connection drops. Wrap in a retry loop for production robustness:

```
const string address = "AA:BB:CC:DD:EE:FF";
const int retryDelayMs = 3000;

while (!cts.Token.IsCancellationRequested)
{
    try
    {
        Console.WriteLine("Connecting to MindWave Mobile...");
        await sdk.ConnectAsync(address, TransportMode.Auto, cts.Token);
        Console.WriteLine("Connected. Streaming data...");

        await foreach (var data in sdk.DataStream(cts.Token))
        {
            ProcessData(data);
        }
```

```
        // If we reach here, DataStream ended without exception
        // (e.g., the headset was turned off)
        Console.WriteLine("Stream ended. Device may have been turned off.");
    }
    catch (OperationCanceledException) when (cts.Token.IsCancellationRequested)
    {
        // User pressed Ctrl+C — exit cleanly
        Console.WriteLine("Stopped by user.");
        break;
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error: {ex.Message}");
        Console.WriteLine($"Reconnecting in {retryDelayMs / 1000} seconds...");
    }

    try
    {
        await Task.Delay(retryDelayMs, cts.Token);
    }
    catch (OperationCanceledException)
    {
        break;  // User cancelled during the retry delay
    }
}
```

Handling NoSignal without disconnecting

The headset may remain connected (Bluetooth link stays up) even when the electrode is not touching the skin. In this case, `SignalQuality` becomes `NoSignal` but `DataStream` keeps running. Handle this in your data processing:

```
await foreach (var data in sdk.DataStream(cts.Token))
{
    if (data.SignalQuality == SignalQuality.NoSignal)
    {
        // Do not process data — log or notify user instead
        UpdateUI("No signal — please adjust the headset.");
        continue;
    }

    ProcessData(data);
}
```

---

13. Advanced Patterns

WPF application with MVVM

```
public class EegViewModel : INotifyPropertyChanged, IAsyncDisposable
{
    private readonly NeuroSkySdk _sdk = new();
    private CancellationTokenSource _cts = new();

    private int _attention;
    private int _meditation;
    private string _status = "Disconnected";

    public int Attention
    {
        get => _attention;
        private set { _attention = value; OnPropertyChanged(); }
    }

    public int Meditation
    {
        get => _meditation;
        private set { _meditation = value; OnPropertyChanged(); }
    }

    public string Status
    {
        get => _status;
        private set { _status = value; OnPropertyChanged(); }
    }

    public async Task ConnectAsync(string macAddress)
    {
        _sdk.StateChanged += (_, state) =>
            App.Current.Dispatcher.Invoke(() => Status = state.ToString());

        await _sdk.ConnectAsync(macAddress);
        await _sdk.SendCommandAsync(NeuroSkyCommand.Notch60Hz);

        _ = Task.Run(async () =>
        {
            await foreach (var data in _sdk.DataStream(_cts.Token))
            {
                if (data.SignalQuality == SignalQuality.NoSignal) continue;

                App.Current.Dispatcher.Invoke(() =>
                {
                    Attention  = data.Attention;
                    Meditation = data.Meditation;
                });
            }
        });
    }

    public event PropertyChangedEventHandler? PropertyChanged;
```

```
    private void OnPropertyChanged([CallerMemberName] string? name = null)
        => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));

    public async ValueTask DisposeAsync()
    {
        _cts.Cancel();
        await _sdk.DisposeAsync();
    }
}
```

Buffering 1 second of raw EEG for custom FFT

```
// Raw EEG arrives as 10 samples per packet.
// Buffer 512 samples to get exactly 1 second of data at 512 Hz.

var buffer = new List<int>(512);

await sdk.SendCommandAsync(NeuroSkyCommand.StartRawEeg);

await foreach (var data in sdk.DataStream(cts.Token))
{
    if (data.RawEeg.Count == 0) continue;  // skip non-raw packets

    buffer.AddRange(data.RawEeg);

    if (buffer.Count >= 512)
    {
        var oneSecond = buffer.GetRange(0, 512).ToArray();
        buffer.RemoveRange(0, 512);

        // Run your own FFT or signal processing here
        var spectrum = MyFft.Compute(oneSecond, sampleRate: 512);
        DisplaySpectrum(spectrum);
    }
}
```

Recording session data to CSV

```
var filename = $"eeg_{DateTime.Now:yyyyMMdd_HHmmss}.csv";

await using var writer = new StreamWriter(filename);
await writer.WriteLineAsync(
    "timestamp_ms,attention,meditation,poor_signal,signal_quality," +
    "delta,theta,low_alpha,high_alpha,low_beta,high_beta,low_gamma,mid_gamma");

await foreach (var d in sdk.DataStream(cts.Token))
{
    // Skip packets with no eSense data (raw EEG-only packets)
    if (d.Attention == 0 && d.Meditation == 0) continue;
```

```
    await writer.WriteLineAsync(
        $"{d.Timestamp},{d.Attention},{d.Meditation},{d.PoorSignal},{d.SignalQuality}," +
        $"{d.Delta},{d.Theta},{d.LowAlpha},{d.HighAlpha}," +
        $"{d.LowBeta},{d.HighBeta},{d.LowGamma},{d.MidGamma}");
}

Console.WriteLine($"Session saved to {filename}");
```

Using Channel for producer/consumer separation

If your processing is heavy and you don't want to block the data stream, use a `Channel<BrainWaveData>` to decouple reading and processing:

```
var channel = Channel.CreateBounded<BrainWaveData>(capacity: 100);

// Producer: reads from BLE and writes to channel
var producer = Task.Run(async () =>
{
    await foreach (var data in sdk.DataStream(cts.Token))
        await channel.Writer.WriteAsync(data, cts.Token);

    channel.Writer.Complete();
});

// Consumer: reads from channel and does heavy processing
var consumer = Task.Run(async () =>
{
    await foreach (var data in channel.Reader.ReadAllAsync(cts.Token))
        await HeavyProcessingAsync(data);
});

await Task.WhenAll(producer, consumer);
```

---

14. Finding Your Device MAC Address

Before calling `ConnectAsync`, you need the Bluetooth MAC address of your MindWave Mobile headset. The MAC address is a 12-character hexadecimal identifier in the format `AA:BB:CC:DD:EE:FF`.

Method 1 — Windows Settings (easiest)

1. Turn on the MindWave Mobile headset (power switch on the left side)
2. Open **Settings → Bluetooth & other devices**
3. If the device is already paired, it appears in the list. Click on "**MindWave Mobile**"
4. Click **"More info"** or look for **"Properties"** in the context menu
5. The MAC address is shown in the device properties as a 12-digit hex string

Method 2 — PowerShell (for already-paired devices)

```
# Lists paired Bluetooth devices whose name contains "MindWave"
Get-PnpDevice -Class Bluetooth |
    Where-Object { $_.FriendlyName -like "*MindWave*" } |
    Select-Object FriendlyName, DeviceID
```

Sample output:

```
FriendlyName        DeviceID
------------        --------
MindWave Mobile     BTHENUM¥...¥7&3A1B2C3D&0&AABBCCDDEEFF_C00000000
```

The last 12 hex characters before `_C00000000` are your MAC address. Format them as `AA:BB:CC:DD:EE:FF`.

Method 3 — Bluetooth LE Explorer app (for BLE scanning)

If the device is not yet paired and you want to scan for its MAC address without pairing:

1. Install **Bluetooth LE Explorer** from the Microsoft Store (free, official Microsoft tool)
2. Open the app and click **Start**
3. Turn on your MindWave Mobile headset
4. Look for a device named "MindWave Mobile" in the scan results
5. The address shown is your MAC address

Method 4 — From your application at runtime

If you want your application to discover the device automatically (without the user entering a MAC address), you can implement a BLE scan using the same WinRT APIs:

```
// Example: scan for MindWave Mobile devices automatically
using Windows.Devices.Bluetooth.Advertisement;

var watcher = new BluetoothLEAdvertisementWatcher();
watcher.Received += (_, args) =>
{
    var name = args.Advertisement.LocalName;
    if (name.Contains("MindWave", StringComparison.OrdinalIgnoreCase))
    {
        // args.BluetoothAddress is a ulong — convert to MAC string:
        var addr = args.BluetoothAddress;
        var mac = string.Join(":", BitConverter.GetBytes(addr)
            .Take(6).Reverse()
            .Select(b => b.ToString("X2")));
        Console.WriteLine($"Found: {name} at {mac}");
    }
};
watcher.Start();
```

---

15. Troubleshooting

Connection issues

Symptom	Likely cause	Solution
`ConnectionState.Error` immediately on BLE	No BLE adapter found	Open Device Manager → Bluetooth → confirm adapter is present and enabled
`Auto` mode always waits 5 seconds before connecting	BLE attempt fails silently	Use `TransportMode.BtClassic` if you've already paired, or check BLE adapter
`BtClassic` fails with access denied or not found	Device not paired	Pair via Settings → Bluetooth & other devices first
`ConnectAsync` never returns (hangs indefinitely)	Bluetooth adapter frozen	Disable and re-enable Bluetooth adapter in Device Manager
Connection succeeds but `DataStream` yields no items	Handshake not sent	This is an SDK internal issue; file a bug report on GitHub

Signal quality issues

Symptom	Likely cause	Solution
`SignalQuality.NoSignal` immediately after connecting	Sensor not touching skin	Adjust headset so the sensor presses firmly on the forehead
`SignalQuality.Poor` persists after 30 seconds	Dry or dirty sensor	Wet the sensor tip with water; clean the forehead
Signal is `Good` but attention/meditation always 0	eSense needs time to warm up	Wait 20~30 seconds after achieving good signal
Constant 60Hz spike in raw EEG	Power-line interference, no notch filter	Call `SendCommandAsync(NeuroSkyCommand.Notch60Hz)` immediately after connecting
Sudden `NoSignal` during use	Headset slipped or ear clip disconnected	Re-seat headset and clip; consider a headband to hold in place

Build and runtime issues

Symptom	Likely cause	Solution
`CS0246: 'NeuroSkySdk' not found`	Missing `using NeuroSky.Sdk;`	Add the using directive to your file
`PlatformNotSupportedException` at runtime	Wrong TargetFramework	Ensure `.csproj` has `net8.0-windows10.0.19041.0`
`FileNotFoundException: WinRT.Runtime.dll`	Package not restored	Run `dotnet restore`

Build succeeds but device not found	MAC address wrong	Double-check MAC address using PowerShell or BLE Explorer
`RawEeg` is always empty	`StartRawEeg` not called	Call `SendCommandAsync(NeuroSkyCommand.StartRawEeg)` after connecting

---

16. Testing

The SDK ships with a unit test suite for `ThinkGearParser` — the packet parser that runs identically on both BLE and BT Classic transports. These tests require no hardware or Bluetooth adapter.

Running the tests

```
dotnet test NeuroSky.Tests/NeuroSky.Tests.csproj
```

Or from the solution root:

```
dotnet test
```

What is covered

Test	Description
`ParseESense_0xEA`	Attention, Meditation, PoorSignal extraction
`ParseESense_0xEA_TooShort`	Short packet → returns null
`ParseESense_0xEB`	Delta, Theta, LowAlpha, HighAlpha extraction
`ParseESense_0xEC`	LowBeta, HighBeta, LowGamma, MidGamma extraction
`ParseRawEeg_Returns10Samples`	20-byte raw EEG → 10 signed int samples
`ParseRawEeg_SignedConversion`	Values > 32768 converted to negative
`ParseRawEeg_TooShort`	Short packet → returns null
`Parse_UnknownUuid`	Unknown UUID → returns null
`ParseByte_ValidPacket`	BT Classic serial packet — Attention/Meditation
`ParseByte_InvalidChecksum`	Wrong checksum → returns null
`ParseByte_PoorSignalCode`	BT Classic PoorSignal (code 0x02)
`SignalQuality_Thresholds`	200/100/25/0 → NoSignal/Poor/Fair/Good

Test location

```
NeuroSky.Tests/
└── ThinkGearParserTests.cs
```

---

17. API Reference

`NeuroSkySdk`

The main entry point. Manages transport selection and lifecycle.

```
public sealed class NeuroSkySdk : IAsyncDisposable
```

Member	Type	Description
`State`	`ConnectionState`	Current connection state (property, get-only)
`StateChanged`	`event EventHandler<ConnectionState>`	Fires whenever the state changes
`ConnectAsync(string, TransportMode, CancellationToken)`	`Task`	Initiate connection to a MindWave Mobile headset
`DisconnectAsync()`	`Task`	Gracefully disconnect the active transport
`DataStream(CancellationToken)`	`IAsyncEnumerable<BrainWaveData>`	Infinite async stream of EEG packets
`SendCommandAsync(byte)`	`Task`	Send a control byte to the headset
`DisposeAsync()`	`ValueTask`	Disconnect and release all Bluetooth resources

`BrainWaveData`

Immutable data record emitted by `DataStream()`.

Property	Type	Range	Description
`Timestamp`	`long`	Unix ms (UTC)	Time this packet was received
`PoorSignal`	`int`	0~200	0 = perfect contact, 200 = no contact
`Attention`	`int`	0~100	eSense™ attention level (0 = not computed)
`Meditation`	`int`	0~100	eSense™ meditation level (0 = not computed)
`Delta`	`int`	0~∞	Delta band power, 0.5~2.75 Hz
`Theta`	`int`	0~∞	Theta band power, 3.5~6.75 Hz
`LowAlpha`	`int`	0~∞	Low Alpha band power, 7.5~9.25 Hz
`HighAlpha`	`int`	0~∞	High Alpha band power, 10~11.75 Hz
`LowBeta`	`int`	0~∞	Low Beta band power, 13~16.75 Hz

`HighBeta`	`int`	0~∞	High Beta band power, 18~29.75 Hz
`LowGamma`	`int`	0~∞	Low Gamma band power, 31~39.75 Hz
`MidGamma`	`int`	0~∞	Mid Gamma band power, 41~49.75 Hz
`RawEeg`	`IReadOnlyList<int>`	-32768~32767	Raw ADC samples at 512 Hz (10 per packet)
`EyeBlink`	`int`	0~255	Eye blink intensity; 0 = no blink detected
`SignalQuality`	`SignalQuality`	enum	Derived from PoorSignal

`TransportMode`

Controls which Bluetooth protocol `ConnectAsync` uses.

Value	Behavior	Pairing required?
`Auto`	BLE first; falls back to BT Classic after 5 seconds	No (BLE path) / Yes (BT Classic fallback)
`Ble`	BLE GATT only	No
`BtClassic`	RFCOMM SPP only	Yes

`ConnectionState`

Reflects the current lifecycle state of the active transport.

Value	Meaning
`Disconnected`	No active Bluetooth connection
`Scanning`	Scanning the Bluetooth adapter for the target device
`Connecting`	Device found, establishing GATT/RFCOMM connection
`Connected`	Connection established, data stream is active
`Error`	Connection attempt failed; inspect exception for details

---

`SignalQuality`

Derived from `BrainWaveData.PoorSignal`. Use this enum in your application logic instead of comparing `PoorSignal` integers directly.

Value	PoorSignal	Reliability	Recommended action
`Good`	0	Full	Use all data
`Fair`	1~50	Acceptable	Use data; minor noise present
`Poor`	51~199	Unreliable	Prompt user to adjust headset
`NoSignal`	200	No data	Prompt user to put on headset

`NeuroSkyCommand`

Static byte constants for `SendCommandAsync`. All commands are sent to the MindWave Mobile handshake characteristic (BLE) or serial output (BT Classic).

Constant	Byte	When to use
`Notch60Hz`	`0x1C`	Power grid is 60 Hz (Korea, USA, Canada)
`Notch50Hz`	`0x1B`	Power grid is 50 Hz (Europe, China, Australia)
`StartRawEeg`	`0x15`	Enable raw EEG waveform (disabled by default)
`StopRawEeg`	`0x16`	Disable raw EEG waveform
`StartEsense`	`0x17`	Enable Attention/Meditation output (enabled by default)
`StopEsense`	`0x18`	Disable Attention/Meditation output

`SimulatorTransport`

Implements `ITransport`. Generates synthetic `BrainWaveData` at ~1 Hz without any hardware.

```
public class SimulatorTransport : ITransport
{
    public enum Mode { Random, Focused, Relaxed, PoorSignal }
    public void SetMode(Mode mode);
}
```

Method / Property	Description
`SetMode(Mode)`	Change simulation mode; takes effect on the next emitted packet
`ConnectAsync(string, CancellationToken)`	Immediately transitions to `Connected` state
`DataStream(CancellationToken)`	Emits one `BrainWaveData` per second
`SendCommandAsync(byte)`	Accepted but no-op in simulation
`DisconnectAsync()`	Transitions to `Disconnected` and ends the data stream

NeuroSky MindWave Mobile Windows SDK v2.0.0 · Apache License 2.0 github.com/nsk-bci/mindwave-sdk-windows